

Chapter 6: Technical Implementation

AAST Connect Graduation Project

This chapter describes how AAST Connect was implemented as a working platform for students, fresh graduates, and administrators. The system was built with Flutter for the application layer and Supabase for backend services, including PostgreSQL database access, storage, remote procedure calls, and real-time updates. The implementation brings together the main workflows required by the platform: secure login, role-based navigation, opportunity discovery, application submission, document handling, training-hour tracking, administrator review, notifications, and AI-assisted support.

The platform is divided into two main experiences. The first is the student and fresh graduate application, which focuses on opportunity access, profile management, training records, applications, and support. The second is the administrator portal, which allows staff members to manage opportunities, review applications, verify training records, and monitor activity through a dashboard.

6.1 Development Environment

AAST Connect was developed using Flutter and Dart because Flutter provides a single codebase for mobile and web interfaces while still allowing a polished user experience. The backend was implemented using Supabase, which provides a managed PostgreSQL database, storage buckets, real-time subscriptions, and a client SDK that integrates directly with Flutter.

Component	Implementation
Programming language	Dart
Application framework	Flutter
Backend platform	Supabase
Database	PostgreSQL
File storage	Supabase Storage
State management	Provider, with Riverpod used in the administrator portal
Local persistence	Shared Preferences
AI assistant	Gemini 2.5 Flash
Main file packages	file_picker, image_picker
Main UI packages	flutter_screenutil, responsive_builder, lucide_icons, timelines_plus, flutter_slidable
Version control	Git and GitHub

The Flutter application initializes Supabase during startup, loads configuration values, and registers application-wide providers for theme handling, chat state, profile data, and opportunity data. A mobile-first layout was used for the user application, with responsive sizing and a constrained web layout so that the same interface remains usable when deployed as Flutter Web.

6.2 System Structure

The implementation follows a modular structure. Instead of placing all logic inside screens, the code is separated into screens, widgets, services, providers, and data models. This makes the project easier to maintain and allows repeated features, such as document cards, navigation bars, status chips, and profile modals, to be reused across the application.

The student side includes screens for the home page, opportunities, profile, tracking, notifications, and support. The fresh graduate side uses a similar structure but with graduate-specific screens and widgets, such as career progress cards, job cards, document sections, and application summaries. Shared services handle authentication, vacancy retrieval, notifications, chat sessions, theme state, and user session data.

The administrator portal uses a more layered structure. It separates remote data sources, repositories, domain use cases, view models, and presentation screens. This structure is useful for the admin module because staff workflows involve several business actions, such as approving applications, rejecting submissions, updating opportunities, retrieving dashboard statistics, and reviewing student records.

6.3 Authentication and Session Handling

AAST Connect uses a custom authentication workflow based on the user's college ID and password. This was necessary because the platform depends on university identity rather than public email-based registration.

In the student and fresh graduate application, the login screen accepts the registration number and password. The identity service sends these values to a PostgreSQL remote procedure call that verifies the login information and returns the user's ID, name, role, and college ID. After a successful login, this information is stored in a

shared session object so that the rest of the application can identify the active user.

The login result controls navigation:

- Students are directed to the student navigation flow.
- Fresh graduates are directed to the fresh graduate navigation flow.
- Administrators are handled through the separate administrator portal.

The administrator portal uses its own staff login workflow. Admin credentials are verified through a database RPC function, and a session context is stored after login. When the admin application resumes, the session context is sent again to the database. This helps database functions identify the active admin and apply the correct access rules during protected operations.

6.4 Role-Based Access Control

Role-based access control is applied in both the interface and the backend. The application does not simply hide buttons; it routes each user into a role-specific experience and retrieves data according to that role.

Students can manage their profiles, browse student-targeted opportunities, save programs, apply to opportunities, upload documents, submit training records, and track their training progress. Fresh graduates can manage graduate profile information, browse graduate-targeted vacancies, submit job applications, upload documents, and track application decisions. Administrators can manage opportunities, review applications, approve or reject training records, inspect student profiles, and view dashboard analytics.

The same database supports all roles, but queries are filtered carefully. For example, student vacancies are filtered by opportunities marked for STUDENT or BOTH, while fresh graduate vacancies are filtered by GRADUATE or BOTH. This allows one opportunity table to serve different audiences without duplicating records.

6.5 Student Implementation

The student module covers the daily workflow of a student who needs to find training opportunities, submit applications, upload proof documents, and track mandatory training hours.

6.5.1 Student Home Screen

The student home screen retrieves the student's profile and training information from Supabase. It displays a summary of training progress, recent opportunities, and upcoming deadlines. Training progress is calculated using the required and completed training-hour fields stored on the student record.

The progress calculation returns completed hours, required hours, remaining hours, extra hours, completion status, and progress percentage. The percentage is capped at 100 percent so that the visual progress indicator remains stable even if a student completes more than the required number of hours.

6.5.2 Student Opportunities

Student opportunities are loaded from the vacancies table through a vacancy service. The query returns opportunities targeted to students and opportunities shared with both students and graduates. The screen supports browsing, searching, filtering, viewing details, saving opportunities, and applying.

The implementation supports two application paths. Some opportunities are handled internally, meaning the student submits the application directly through AAST Connect. Other opportunities may use an external application link, in which case the app opens the external URL while still presenting the opportunity inside the platform.

6.5.3 Student Applications

When a student applies internally, the system creates a new record in the application table. The application stores the vacancy ID, applicant ID, applicant name, college ID, cover letter, submission date, status, and optional document reference. New applications start with a pending status.

Before submitting, the service checks whether the student has already applied to the same opportunity. Canceled applications are excluded from this check, which allows a user to submit again only when the previous active application is no longer valid.

A key implementation detail is how application documents are handled. When a student selects an existing uploaded document, the system creates a separate copy of that file in Supabase Storage and links the copied file to the application. This prevents a later profile document edit or deletion from changing the document that was submitted with the application.

6.5.4 Training Records

Students can submit training records from the profile workflow. A training record includes the submitted hours, related training information, proof document, and review status. Supporting certificates or proof files are uploaded to Supabase Storage and linked through the document table.

Submitted records remain pending until an administrator reviews them. If the record is approved, the administrator workflow updates both the training record status and the student's completed training hours. If the record is rejected, the rejection reason is stored so the student can understand the decision.

6.5.5 Student Profile and Documents

The student profile connects data from the student, profile, document, and saved program tables. The profile screen allows students to view and update personal details, manage skills and interests, add portfolio links, upload documents, view saved opportunities, and submit training hours.

Document management was implemented with care because the same document may be used in different workflows. Files are uploaded to Supabase Storage, and their URLs are stored in the database. When a document is deleted from the profile, the system checks whether it is already linked to an application. If it is linked, the app avoids breaking the historical application record.

6.6 Fresh Graduate Implementation

The fresh graduate module focuses on career opportunities, profile readiness, document uploads, and application tracking. Although it shares several services with the student module, it uses a separate navigation flow and graduate-specific queries.

6.6.1 Fresh Graduate Profile

Fresh graduate profile data is stored in the fresh graduate table and loaded using the user's college ID. The profile supports biography, phone number, GPA, major, skills, interests, portfolio links, and uploaded documents. A profile provider manages this data so that screens can display updated information without repeating database logic.

Fresh graduates can upload documents to Supabase Storage and link them to profile records. These documents can later be selected during internal applications. As with student applications, selected documents are copied into an application-specific storage path so that each application keeps its own stable submission file.

6.6.2 Graduate Opportunities

Graduate opportunities are retrieved from the vacancies table using graduate-specific filters. The app shows opportunities marked for GRADUATE or BOTH, which keeps the graduate experience focused on relevant vacancies.

Fresh graduates can browse opportunities, view details, apply internally, and track which vacancies they have already applied to. The latest graduate opportunities are also used by the home screen and the chat assistant so that the system can provide relevant suggestions.

6.6.3 Graduate Applications and Tracking

Fresh graduate applications are stored in the same application table used by students. The service resolves the logged-in user, reads the user's name and college ID, and inserts a pending application with the selected vacancy, cover letter, submission date, and optional document snapshot.

The tracking screen displays application status and rejection reasons when available. This gives fresh graduates a simple way to follow their submissions without contacting administrators manually.

6.6.4 Graduate Opportunity Notifications

Fresh graduate opportunity notifications are generated by comparing recent graduate-targeted vacancies with locally stored seen vacancy IDs. The app stores seen IDs using Shared Preferences. When a new graduate opportunity appears and has not been marked as seen, it can be shown as an unread opportunity notification.

This approach keeps the feature lightweight because it does not require creating a separate database notification record for every graduate user whenever a new opportunity is posted.

6.7 Administrator Implementation

The administrator portal provides the staff-facing tools needed to manage the system. It includes dashboard analytics, opportunity management, application review, training record review, student profile inspection, and certificate upload.

6.7.1 Opportunity Management

Administrators can create, update, delete, search, filter, and paginate opportunities. Opportunity records are stored in the vacancies table. Pagination is handled using Supabase range queries, which prevents the application from loading all records at once.

The opportunity screen also retrieves applicant counts from a database view. This allows administrators to see how many users applied to each opportunity without manually counting application records in the interface.

Expired opportunities are filtered using the deadline field, so the admin list focuses on opportunities that are still active.

6.7.2 Application Review

Administrators can review submitted applications and approve or reject them. The approval workflow updates the application status to approved. The rejection workflow updates the status to rejected and stores a rejection reason.

Application data is enriched with related information, such as company name, GPA, and profile image where available. This helps administrators review applications with enough context instead of seeing only raw application records.

6.7.3 Training Record Review

Training records are retrieved through database functions designed for admin access. The admin portal joins each training record with student information so that staff can see the student's name, college ID, current completed hours, submitted hours, and proof documents.

When a training record is approved, the system performs two important updates. First, it adds the submitted hours to the student's completed training hours. Second, it marks the training record as approved. When a record is rejected, the system marks it as rejected and stores the reason.

The admin portal also supports uploading certificates to a certificates storage bucket and saving the certificate URL on the training record.

6.7.4 Dashboard and Student Profiles

The admin dashboard summarizes platform activity. It retrieves application counts, recent applications, recent training uploads, and student names related to training submissions. These values are used to display total applications, pending applications, training uploads, and recent activity cards.

The student profile section uses paginated database functions to retrieve student records. Application counts are also retrieved and mapped to student profiles, allowing administrators to quickly see each student's activity level.

6.8 Database Implementation

Supabase PostgreSQL is the main data layer of the system. The application reads and writes data through Supabase client queries and PostgreSQL RPC functions. RPC functions are used where the operation needs custom verification, protected admin access, aggregated results, or session context.

The main database objects used by the platform include:

- users;
- student;
- freshgraduate;
- profile;
- vacancies;
- application;
- trainingrecord;
- document;
- saved_programs;
- notification;
- student_application_counts;
- vacancy_applicant_counts.

This structure allows the system to keep user identity, profile data, applications, training records, documents, and opportunity records connected while still separating their responsibilities.

6.9 Storage Implementation

Supabase Storage is used for uploaded files. The user application stores CVs, certificates, training proof files, and general profile documents in the documents bucket. The administrator portal also uses a certificates bucket for certificate uploads related to training records.

Each uploaded file is linked to a database record, usually through the document table or a training record certificate field. The application stores the public storage URL so that the file can be displayed or downloaded later.

The application document snapshot mechanism improves reliability. Instead of linking an application directly to a profile document that may later change, the system creates a copy of the selected document when the application is submitted. This gives each submitted application its own document history.

6.10 Notification Implementation

Notifications are implemented as in-app features. Student notifications are stored in the notification table. The notification service can fetch notifications, count unread items, mark one or all notifications as read, delete notifications, and listen for new notification records through Supabase real-time subscriptions.

The notification service also formats timestamps and chooses an icon and color based on the message content. For example, accepted or approved messages are shown differently from rejected messages or deadline

reminders.

Fresh graduate opportunity notifications use a lighter approach. They are generated from recent graduate vacancies and local seen-state. This is suitable for opportunity alerts because the app only needs to know whether the current user has already seen a posted vacancy.

6.11 AI Chat Assistant

AAST Connect includes an AI assistant built with Gemini 2.5 Flash. The assistant is designed to help students and fresh graduates understand their training hours, applications, documents, opportunities, and profile status.

Before sending a question to Gemini, the app builds a private context from live Supabase data. For students, this context includes required training hours, completed training hours, recent applications, and recent student opportunities. For fresh graduates, it includes profile information, recent applications, rejection reasons, and latest graduate opportunities.

The assistant uses this context to answer naturally without exposing the internal database structure to the user. The system prompt also limits the assistant to student and fresh graduate support. It avoids administrator features and changes its answers according to the logged-in user's role.

To reduce repeated database calls, the chat provider caches the context for a short period. The cache is cleared when the user's data may have changed, such as after submitting hours or refreshing the application state.

6.12 Performance and Reliability

Several implementation choices improve performance and reliability:

- paginated admin opportunity retrieval;
- paginated admin student retrieval;
- database views for application and applicant counts;
- limited latest-opportunity queries for home screens and chat context;
- cached chat context;
- local seen-state for graduate opportunity notifications;
- real-time subscriptions for student notifications;
- detached document copies for submitted applications.

These choices reduce unnecessary database traffic and protect important records from accidental changes.

6.13 User Interface Implementation

The interface was built using reusable Flutter widgets and a consistent visual style. The student module includes components for home statistics, deadlines, opportunity cards, filters, application modals, document cards, profile modals, and tracking status indicators. The fresh graduate module includes profile cards, job cards, document sections, career progress cards, status badges, and application summaries.

The administrator interface includes dashboard cards, recent activity cards, opportunity tables, skeleton loaders, student cards, approval screens, and staff navigation. These components support repeated staff tasks and make it easier to scan pending work.

The application also supports light and dark themes, responsive sizing, loading states, form validation messages, and clear status colors for pending, approved, rejected, and canceled records.

6.14 Implementation Challenges

One of the main challenges was implementing college-ID authentication. The system needed to support university identity and role detection, so a custom RPC-based login flow was used instead of a standard email-only flow.

Another challenge was supporting two different user groups in one user-facing application. Students and fresh graduates share some features, such as opportunities, applications, documents, and support, but they also have different needs. Students require training-hour submission and progress tracking, while fresh graduates require job-focused tracking and profile readiness. This was handled through role-based navigation and role-specific queries.

Document handling also required careful design. Users may update or delete profile documents, but submitted applications must remain stable. The document snapshot approach solves this problem by preserving a separate copy of the submitted file.

The administrator training approval workflow was another important area. Approving training does not only change the training record status; it must also update the student's completed hours. This required the approval workflow to update related records in the correct order.

Finally, the AI assistant needed to be useful without becoming too broad. The implementation handles this by giving the model role-specific context and clear instructions about the assistant's scope.

6.15 Summary

The technical implementation of AAST Connect combines Flutter, Supabase, PostgreSQL, Supabase Storage, Provider, Riverpod, Shared Preferences, real-time database subscriptions, and Gemini 2.5 Flash. The result is a role-aware platform that supports students, fresh graduates, and administrators through separate but connected workflows.

Students can manage profiles, browse opportunities, submit applications, upload documents, submit training records, track progress, receive notifications, and use the AI assistant. Fresh graduates can manage career profiles, browse graduate opportunities, apply for vacancies, upload documents, track decisions, and receive opportunity alerts. Administrators can manage opportunities, review applications, verify training records, monitor dashboard statistics, and inspect student profiles.

Overall, the implementation provides the core technical foundation needed for AAST Connect to operate as a centralized academic training and opportunity platform.